University of Applied Sciences

HOCHSCHULE EMDEN·LEER

Machine Learning Pipeline

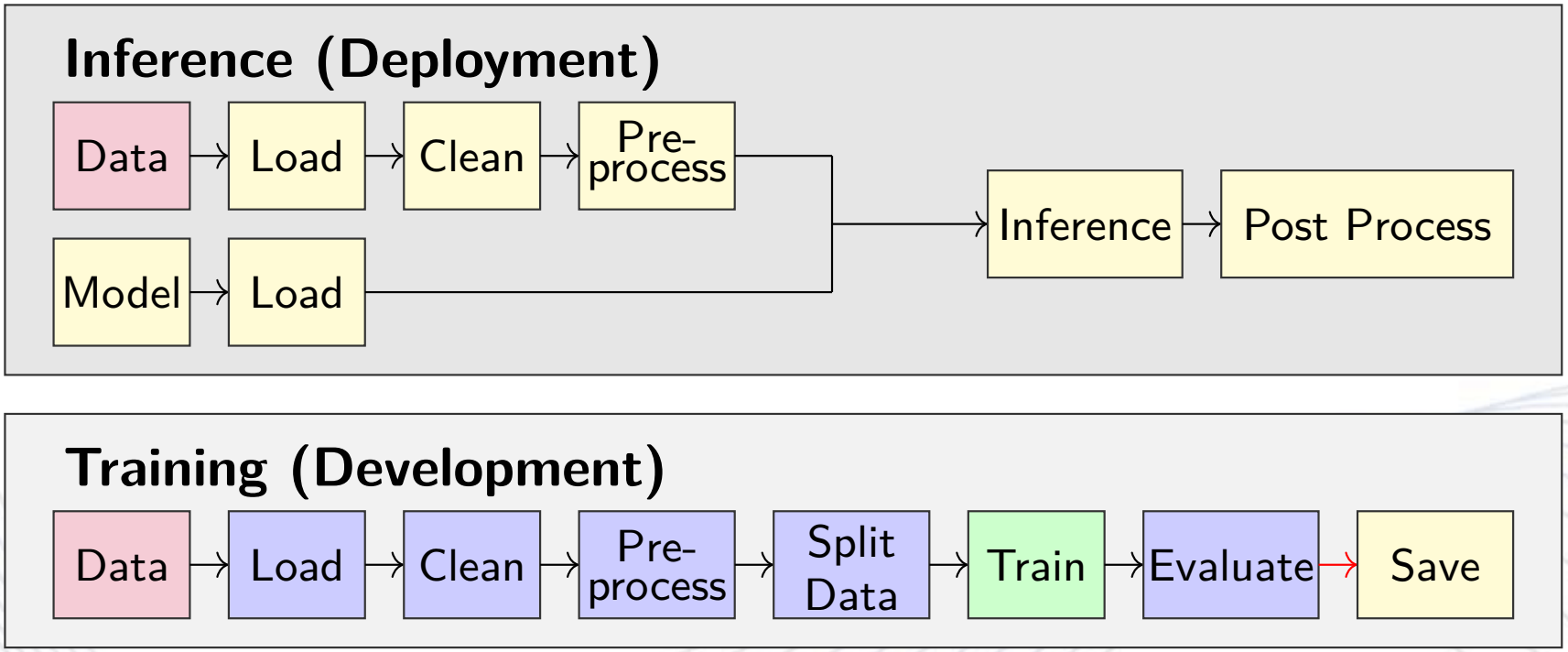
Prof. Dr. E. Wings

Agenda

1. ML Pipeline
2. Dataset
3. Load & Clean & Split Data
4. Pre Process
5. Train
6. Evaluate
7. Save
8. Data
9. Load
10. Clean
11. Pre Process
12. Model
13. Load
14. Inference
15. Post Process
16. Evaluation
17. Iterate on Models

ML Pipeline

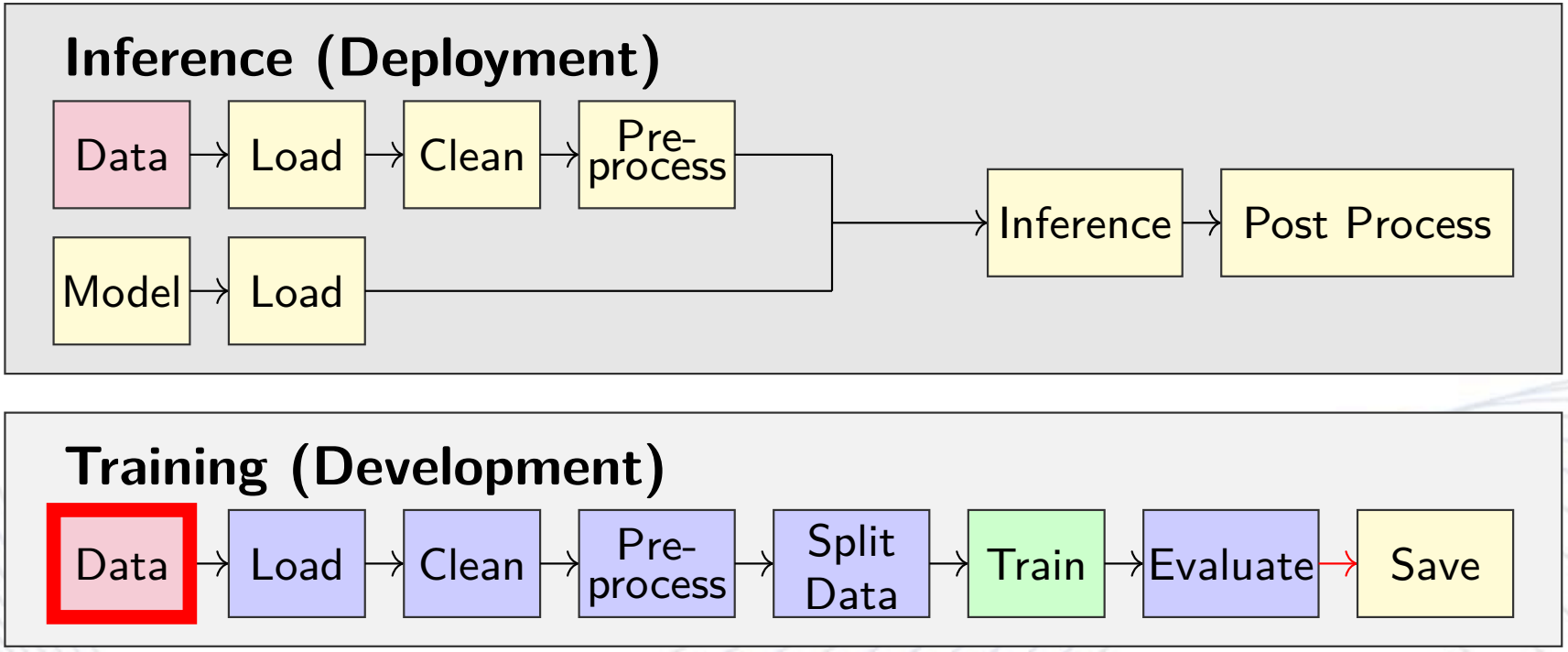
ML Pipeline



Training and inference pipelines are complementary

Dataset

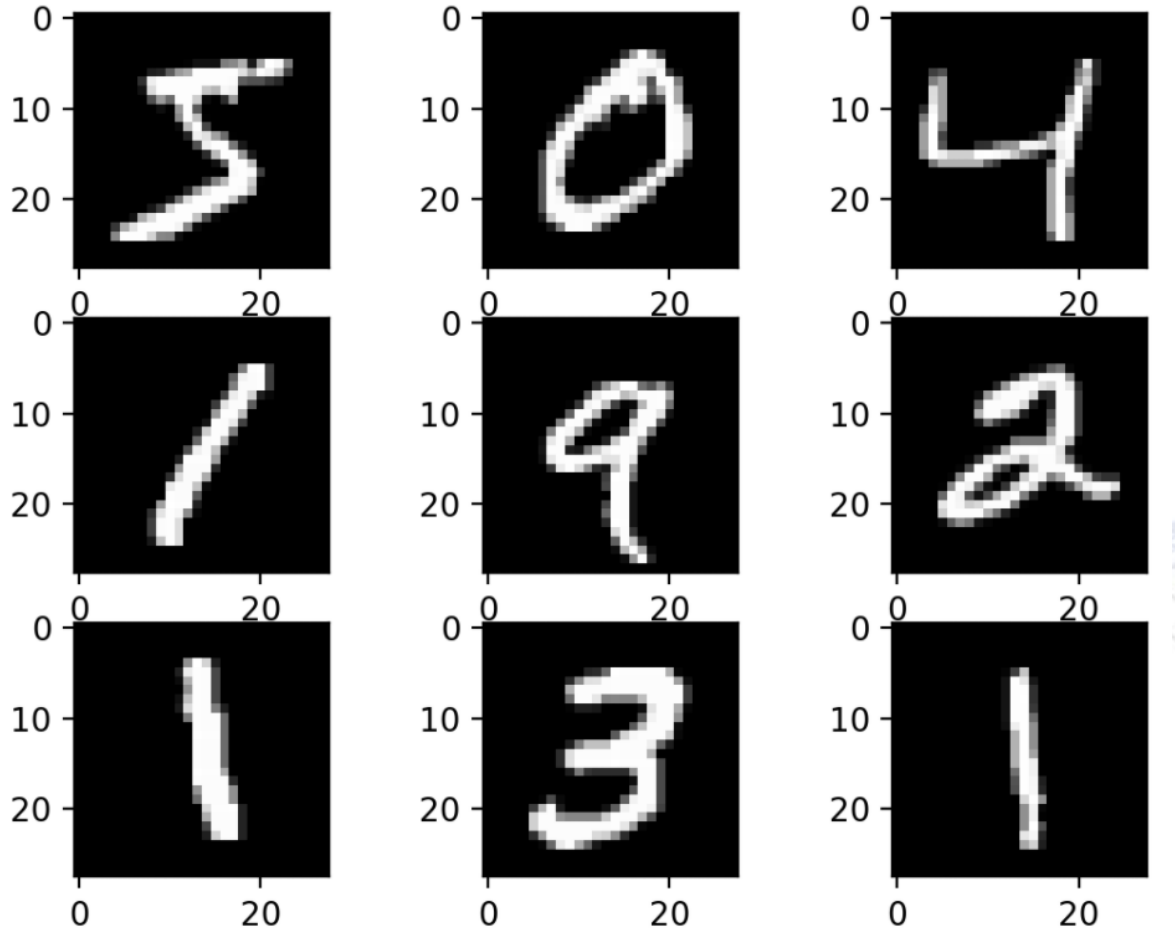
ML Pipeline - Data



Dataset MNIST

- MNIST Handwritten Digit Classification Dataset
- The MNIST dataset is an acronym that stands for the Modified National Institute of Standards and Technology dataset.
- It is a dataset of
  - 60,000 for training
  - 10,000 for testing
  - square 28×28 pixel
  - grayscale images of handwritten single digits
  - classes between 0 and 9.
- The library Keras contains the dataset.

Dataset MNIST



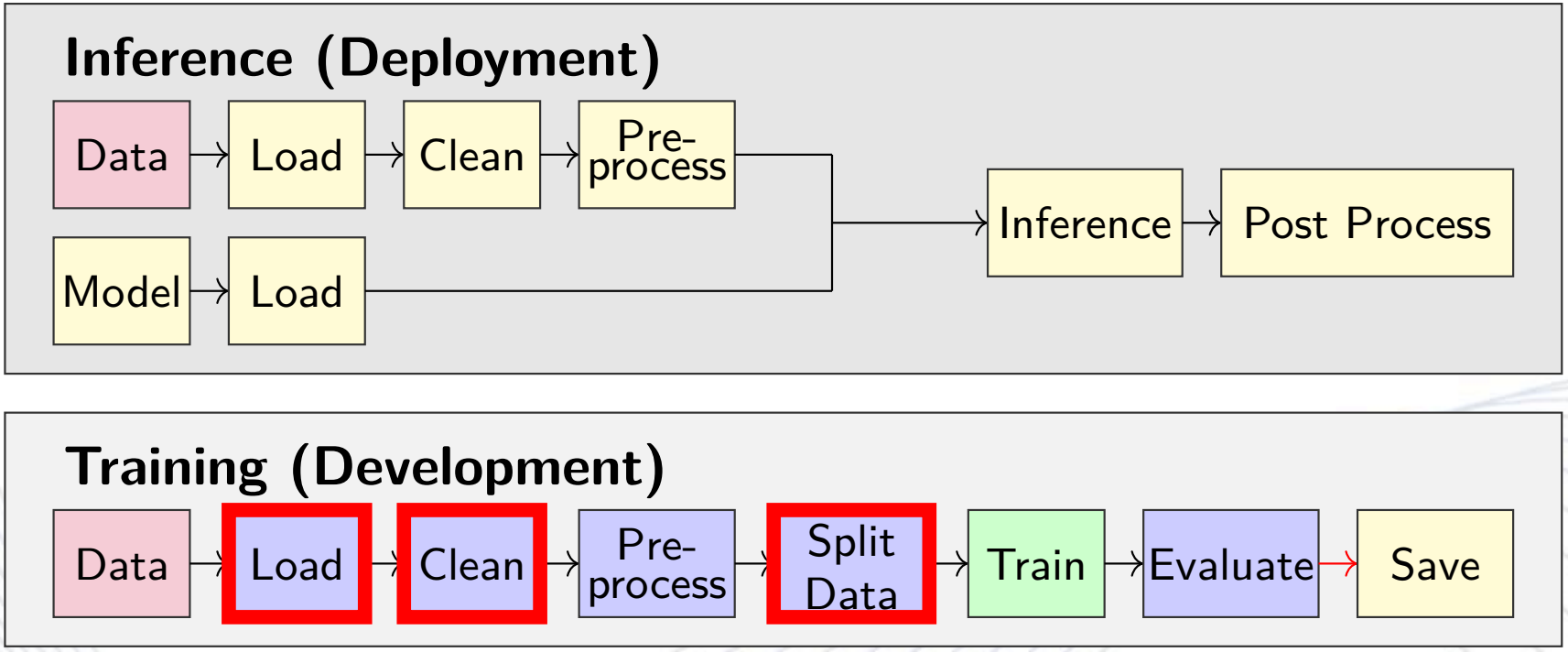
Dataset MNIST

The task is to classify a given image of a handwritten digit into one of 10 classes representing integer values from 0 to 9, inclusively.

- It is a widely used and deeply understood dataset and, for the most part, is "solved".
- Top-performing models are deep learning convolutional neural networks that achieve a classification accuracy of above 99%, with an error rate between 0.4% and 0.2% on the hold out test dataset.

Load & Clean & Split Data

ML Pipeline - Load & Clean & Split Data



Loading MNIST

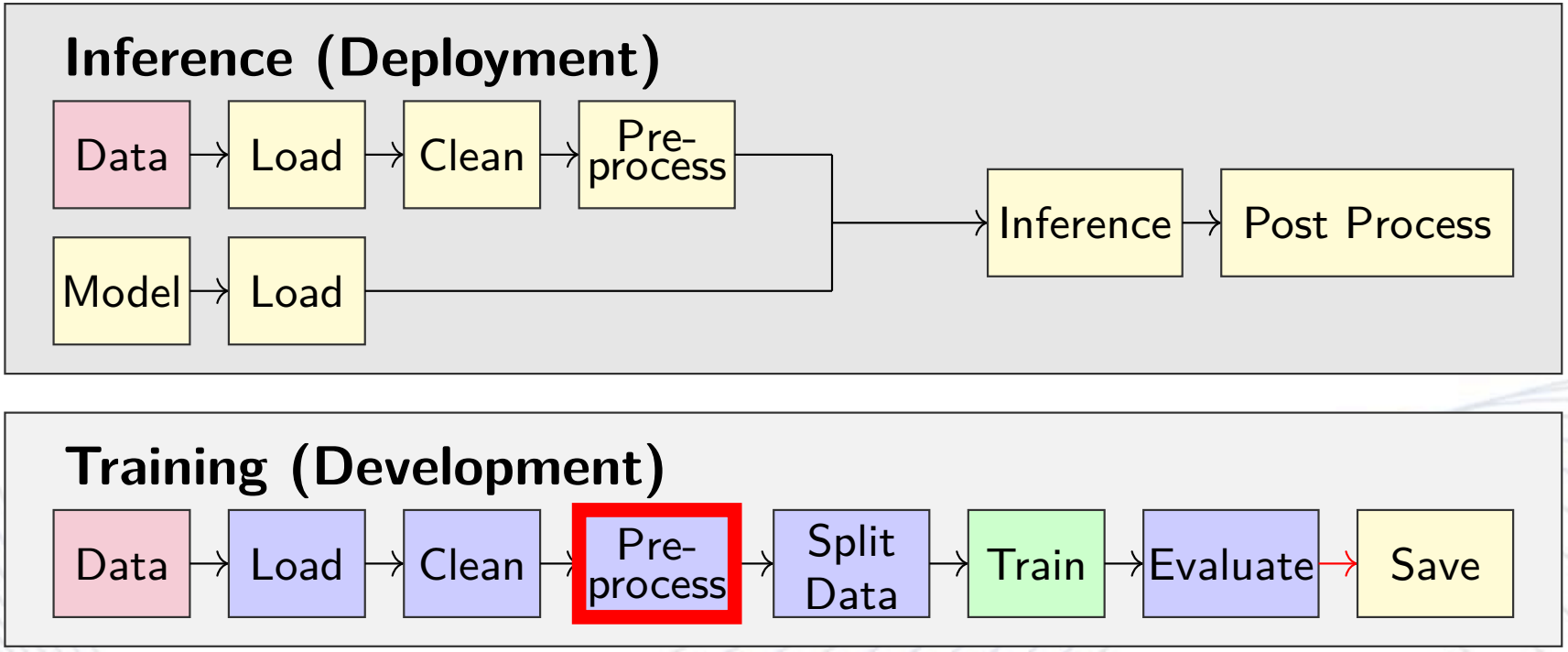
```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

(train_images, train_labels), (test_images, test_labels) = mnist.
    load_data()

print("Shape Trainingsdaten: {}".format(train_images.shape))
print("Dimension Bild Nr. 5: {}".format(train_images[5].shape))
print("Label zu Bild Nr. 5: {}".format(train_labels[5]))
```

Output:

```
Shape Trainingsdaten:  (60000, 28, 28)
Dimension Bild Nr.  5:  (28, 28)
Label zu Bild Nr.  5:  2
```

Pre Process

ML Pipeline - Pre Process



Scaling

```
train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype('float32')
train_images /= 255
test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images.astype('float32')
test_images /= 255

train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
NrTrainimages = train_images.shape[0]
NrTestimages = test_images.shape[0]

print("Training dataset:{}".format(train_images.shape))
print("Test dataset:{}".format(test_images.shape))
print("We have {} train images and {} test images.".format(NrTrainimages,
    NrTestimages))
```

Output:

```
Training dataset:(60000, 28, 28, 1)
Test dataset:(10000, 28, 28, 1)
We have 60,000 train images and 10,000 test images.
```

One-Hot-Encoding

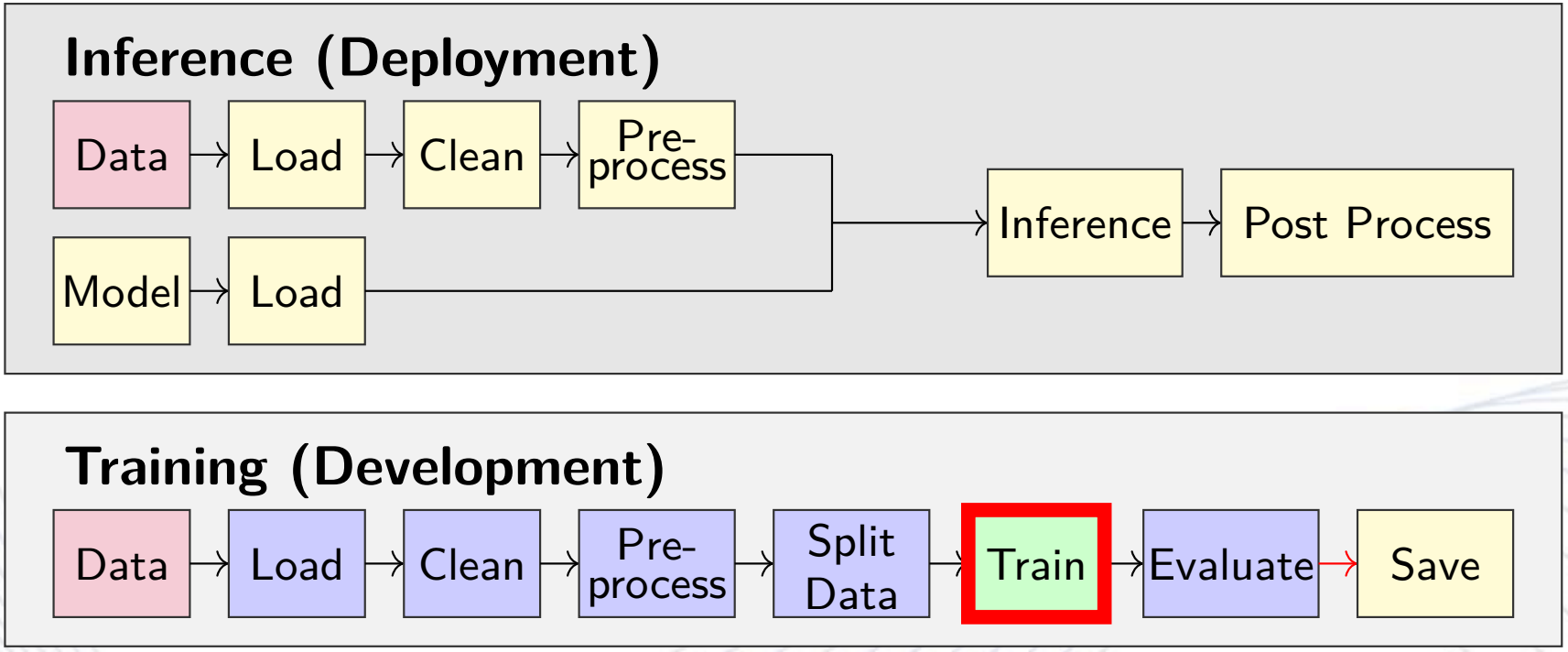
The function `to_categorical()` maps the categorical classes to a vector:

We have 10 classes, so we get e.g.:

class '3' maps to $(0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$

Train

ML Pipeline - Train



Creating CNN - Import Libs

```
import os
import numpy as np

from tensorflow.keras.callbacks import TensorBoard
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras import backend as K
```

Creating CNN - The Model

Now, we specify a model that consists of defining the following elements:

- Network architecture with hyperparameters
- Loss function
- Optimizer
- Model evaluation

Creating CNN - The Model

```
# Das Format der Eingabedaten wird an das erste Layer Ã¼bergeben
mnist_inputshape = train_images.shape[1:4]
# Die Netzwerkarchitektur
model = Sequential()
# Detektionsteil
model.add(Conv2D(32, kernel_size=(5,5),
activation = 'relu',
input_shape=mnist_inputshape))
model.add(MaxPooling2D(pool_size=(2,2)))
# Conv_Block 2
model.add(Conv2D(64, kernel_size=(5,5),activation= 'relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.5))
# Identifikationsteil
model.add(Flatten())
model.add(Dense(128, activation='relu', name='features'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dense(10,activation='softmax'))
model.summary()
```

Creating CNN - The Model

Output:

```
Model:  "sequential"

_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 24, 24, 32)        832
max_pooling2d (MaxPooling2D  (None, 12, 12, 32)        0)
conv2d_1 (Conv2D)            (None, 8, 8, 64)          51264
max_pooling2d_1 (MaxPooling  (None, 4, 4, 64)          0 2D)
dropout (Dropout)            (None, 4, 4, 64)          0
flatten (Flatten)            (None, 1024)              0
features (Dense)             (None, 128)               131200
dropout_1 (Dropout)          (None, 128)               0
dense (Dense)                (None, 64)                8256
dense_1 (Dense)              (None, 10)                650
=================================================================
Total params:  192,202
Trainable params:  192,202
Non-trainable params:  0
_________________________________________________________________
```

Creating CNN - The Model

With `model.summary()`, we get some interesting information about our architecture.

Here, we get nearly 200,000 parameters.

Creating CNN - Compiling the Model

Determination of loss function and optimization:

```
model.compile(loss='categorical_crossentropy',
              optimizer='Adam',
              metrics=['accuracy'])
```

Loss Function and Optimizer

We use the loss function Categorical Cross-Entropy L:

$$L(i) = \sum_{j \in \text{class}} y_{i,j} \cdot \ln(\hat{y}_{i,j})$$

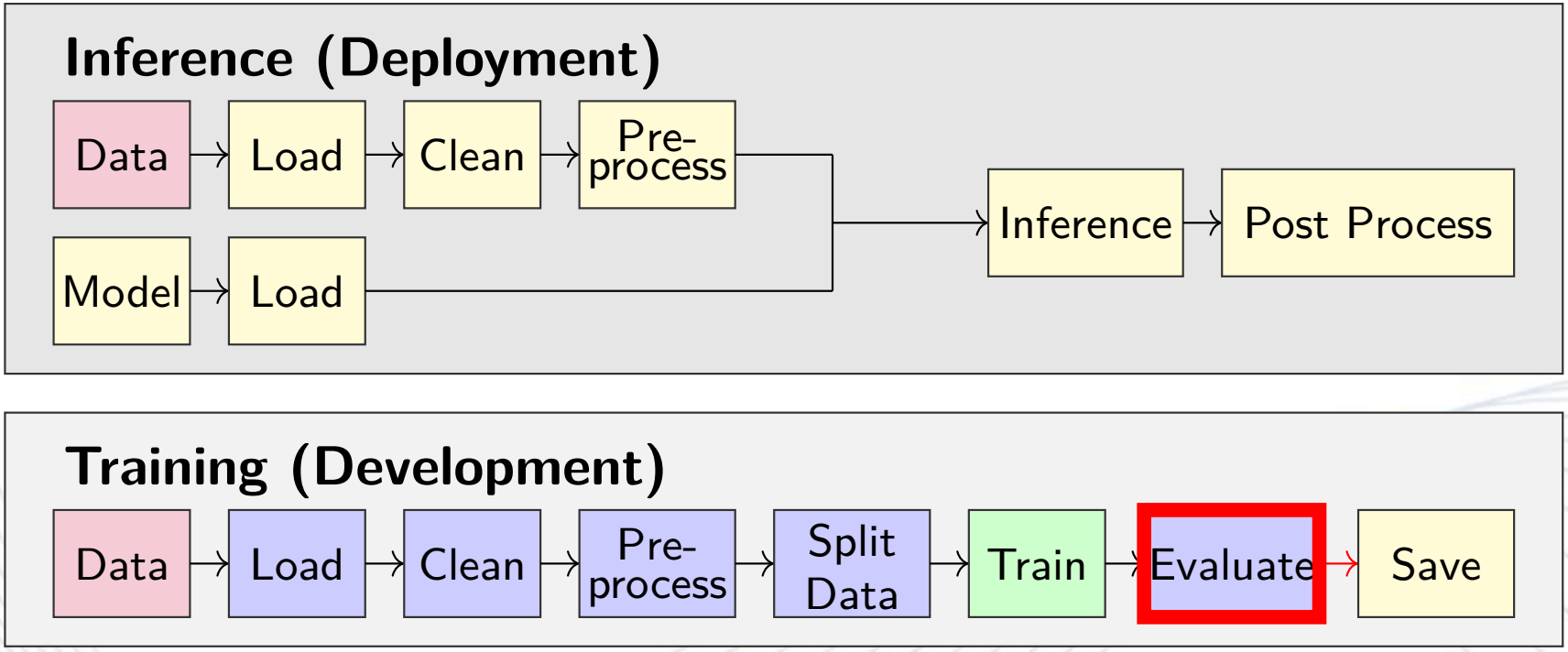
j is the class, $y_{i,j}$ is the goal label 0 or 1, $\hat{y} \in [0; 1]$ is the prediction.

The optimzer is the Adam-optimizer:

The **Ada**ptive **M**oment Estimation takes into account past gradients and additionally their coordinate-wise squared gradients. However, only the average of the previous gradients and the squared gradients is used for the change of the weights.

Evaluate

ML Pipeline - Evaluate



Tensorboard

We want to follow the training:

Launch Tensorboard in the console:
`tensorboard -logdir="logs"`

The result is an url which you have to launch in a browser:

`http://localhost:6006/`

Perhaps, there is another number.

Tensorboard

Create a directory for the history and start the Tensorboard:

```
# TensorBoard - Vorbereitung
LOGDIR = "logs"

my_tensorboard = TensorBoard(log_dir = LOGDIR,
                             histogram_freq=0,
                             write_graph=True,
                             write_images=True)
```

Defining the Hyperparameters and Training

```
# Hyperparameter
my_batch_size = 128
my_num_classes = 10
my_epochs = 12
history = model.fit(train_images, train_labels,
                    batch_size=my_batch_size,
                    callbacks=[my_tensorboard],
                    epochs=my_epochs,
                    verbose=1,
                    validation_data=(test_images, test_labels))
```

Defining the Hyperparameters and Training

out:

```
Epoch 1/12

271/469 [================>............] - ETA: 19s - loss: 0.5346 - accuracy:
     0.8263
```

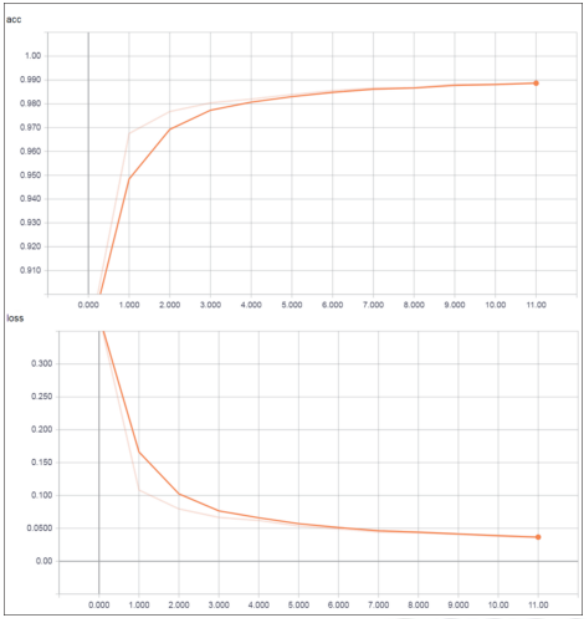
The listing already shows the results for the loss function (loss) and the accuracy (acc) for the training data set and for the test data set

Defining the Hyperparameters and Training

```
score = model.evaluate(test_images, test_labels)
print('Test Verlust:', score[0])
print('Test Genauigkeit:', score[1])
```

We have achieved a very good accuracy of 99.4 %, and this after only a few epochs.

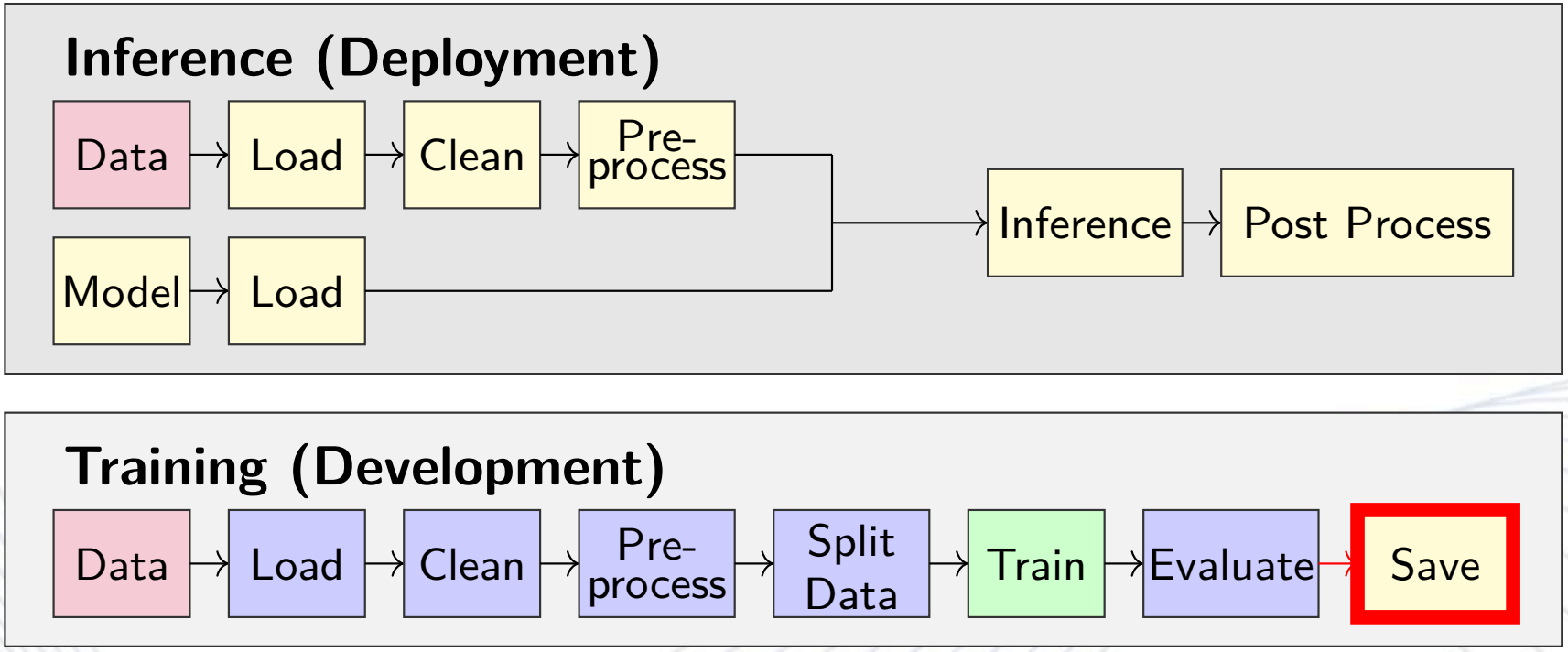
Tensorboard



It would be interesting to look at the loss and accuracy during the training. We show the accuracy ("acc") and the loss ("loss") of the training data and test data respectively over the training epochs.

Save

ML Pipeline - Save



Save

After running this example, you will now have a 1.2-megabyte file with the name `final_model.h5` in your current working directory.

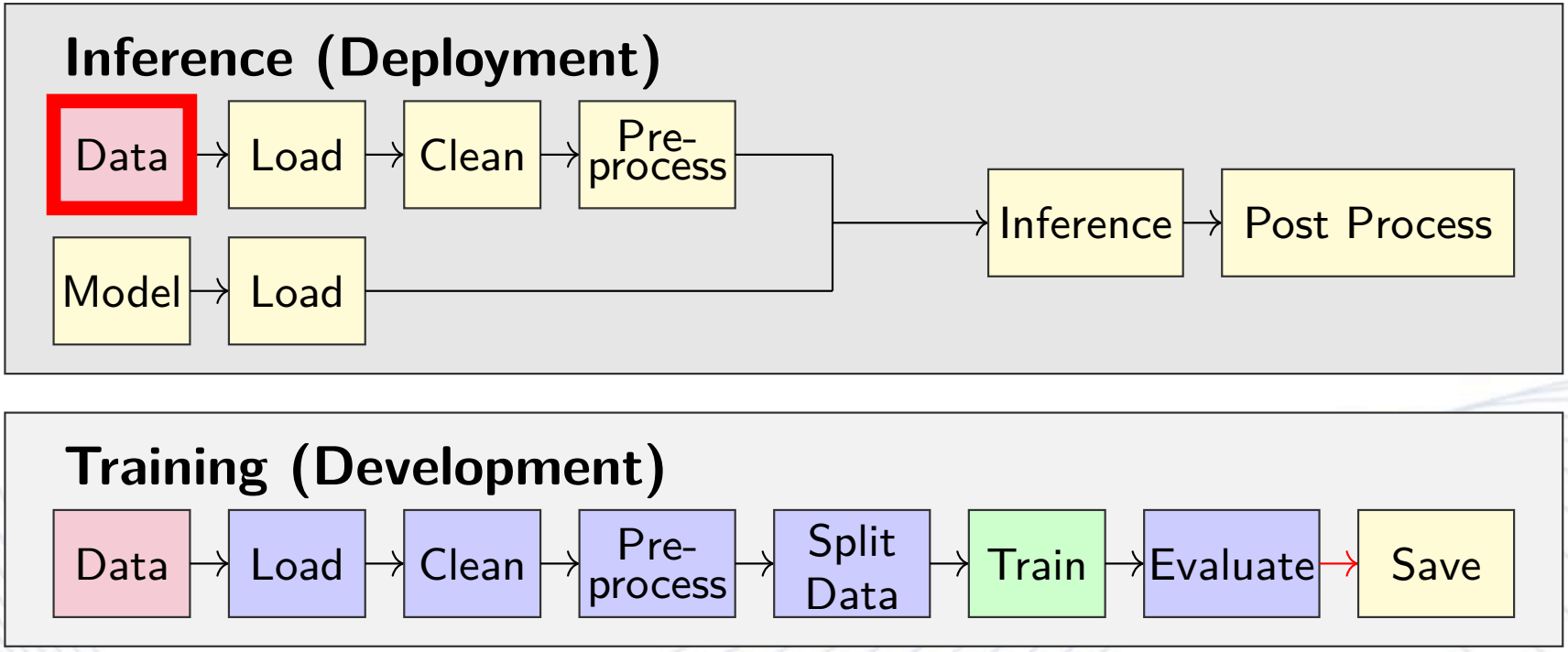
```
# save model
model.save('final_model.h5')
```

Inference

We can use our saved model to make a prediction on new images.

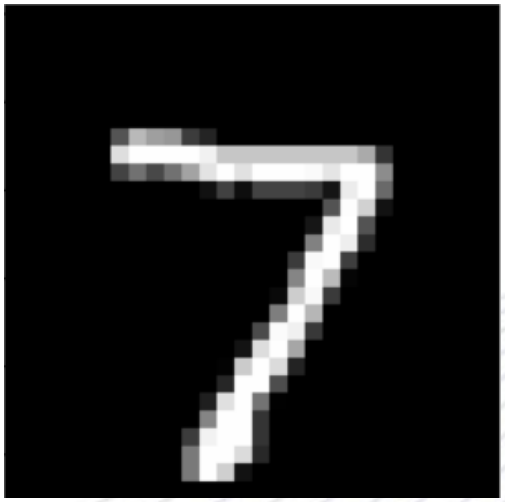
Data

ML Pipeline - Data



ML Pipeline - Data

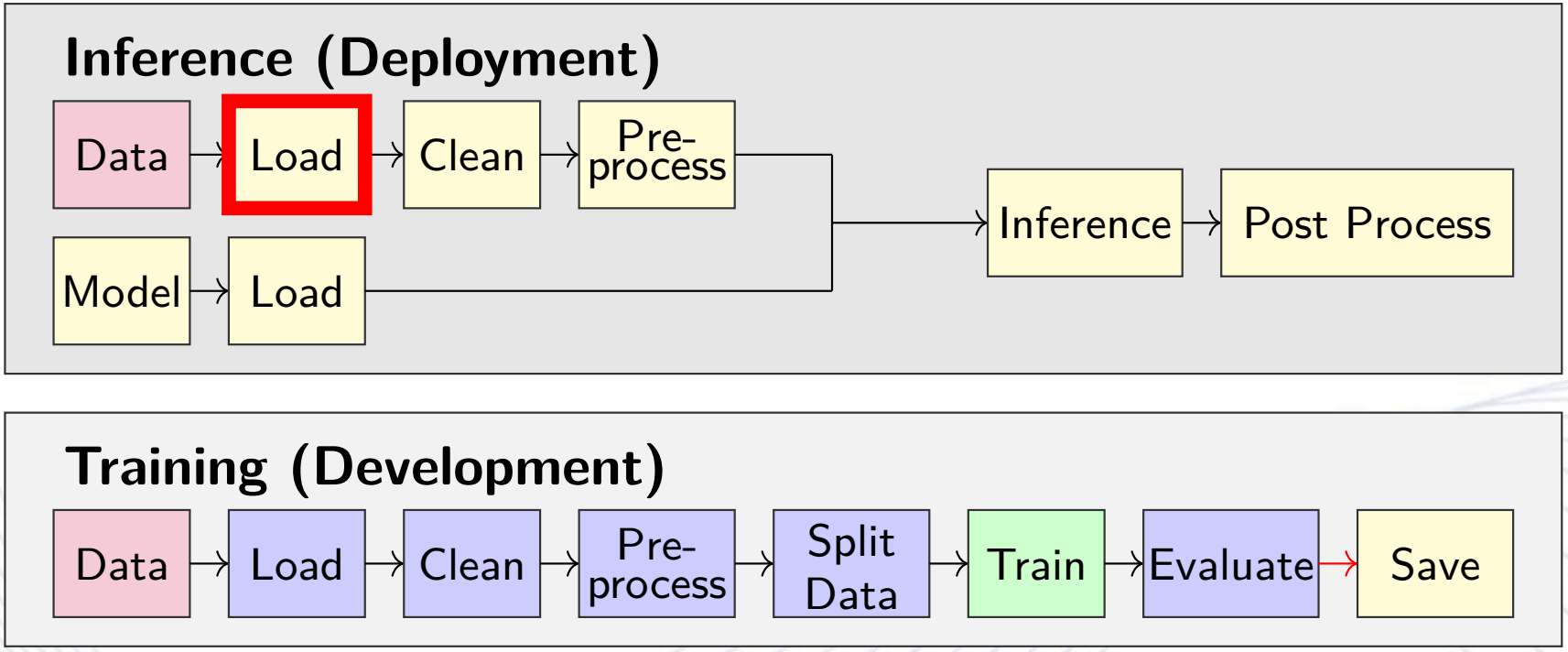
The model assumes that new images are grayscale, that they have been aligned so that one image contains one centered handwritten digit, and that the size of the image is square with the size 28×28 pixels.



The image is extracted from the MNIST test dataset. It is in the current working directory with the filename `sample_image.png`.

Load

ML Pipeline - Load



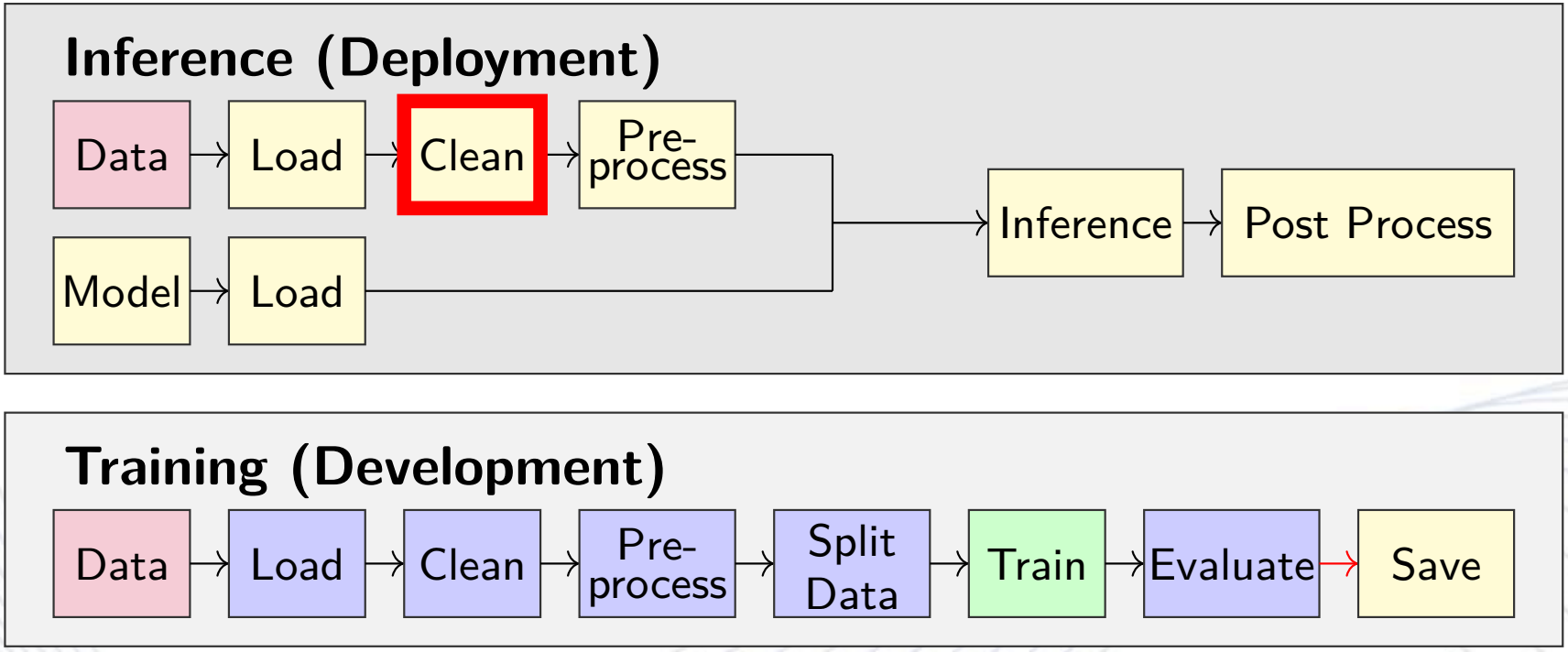
Load

We have to load an image:

```
# load the image
img = load_image('sample_image.png')
```

Clean

ML Pipeline - Clean



ML Pipeline - Clean

The model assumes that new images are grayscale, that they have been aligned so that one image contains one centered handwritten digit, and that the size of the image is square with the size 28×28 pixels.

The image is forced to be in grayscale format, and be forced the size to be 28×28 pixels.

The loaded image can then be resized to have a single channel and represent a single sample in a dataset. The function `load_image()` implements this and will return the loaded image ready for classification.

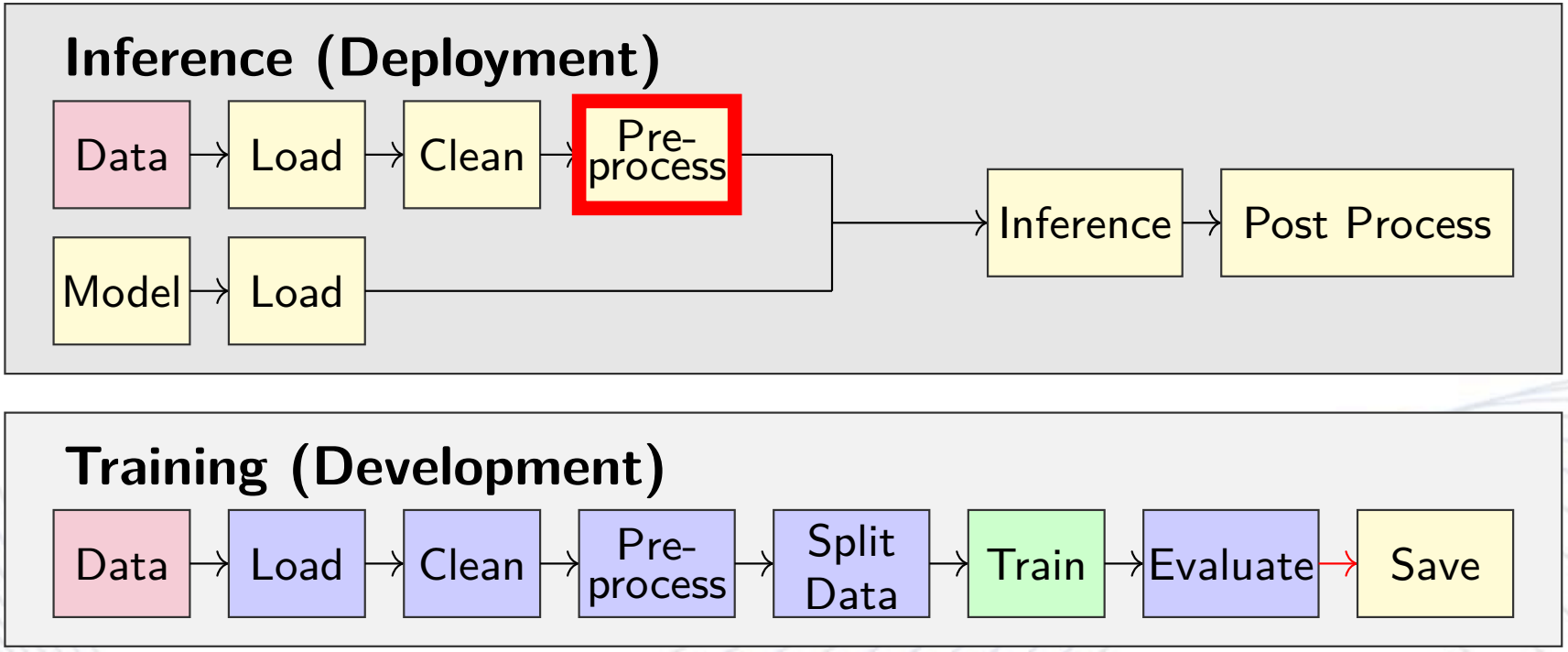
Clean

For interference, you have to laod the model.

```
# load and prepare the image
def load_image(filename):
    # load the image
    img = load_img(filename, grayscale=True, target_size=(28, 28))
    # convert to array
    img = img_to_array(img)
    # reshape into a single sample with 1 channel
    img = img.reshape(1, 28, 28, 1)
```

Pre Process

ML Pipeline - Pre Process



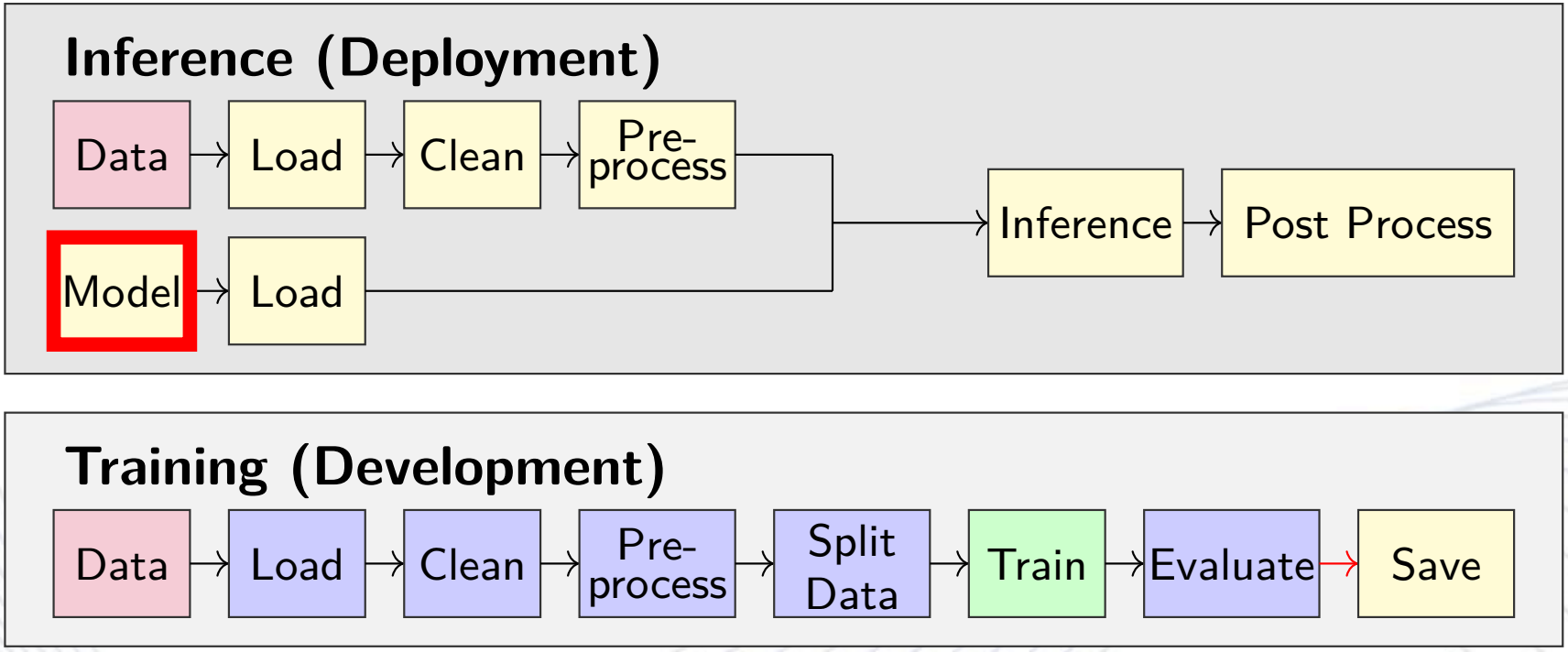
ML Pipeline - Pre Process

Importantly, the pixel values are prepared in the same way as the pixel values were prepared for the training dataset when fitting the final model, in this case, normalized.

```
# prepare pixel data
img = img.astype('float32')
img = img / 255.0
return img
```

Model

ML Pipeline - Model

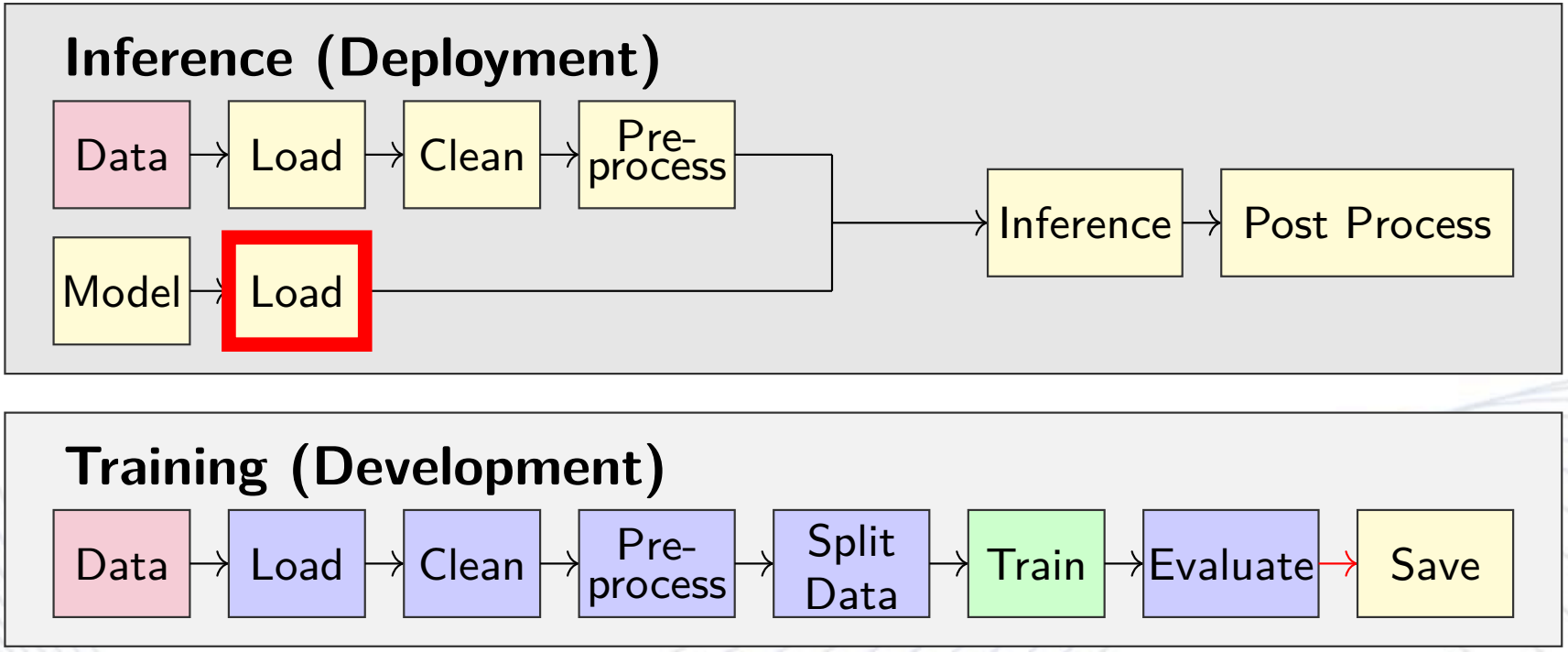


ML Pipeline - Model

We design and develop a model. Now we can use this model for inference.

Load

ML Pipeline - Load



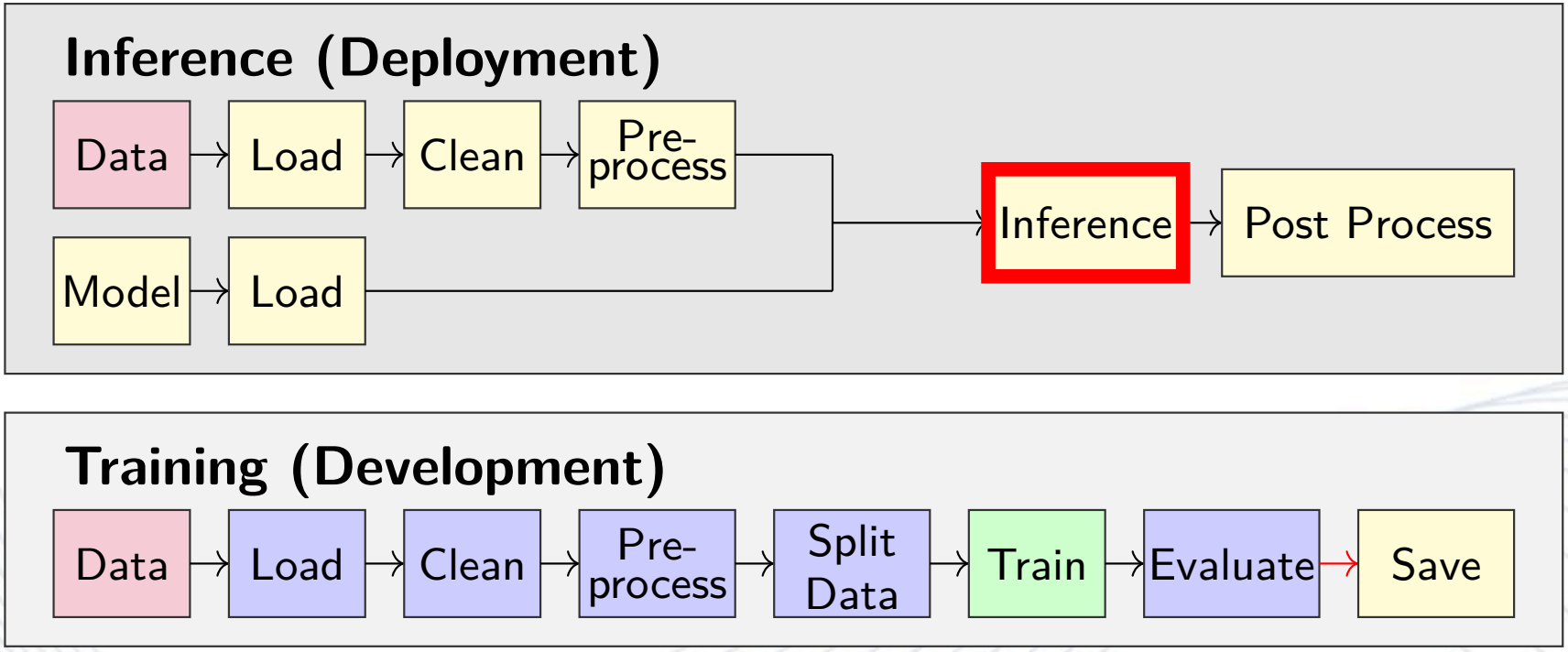
Load

For interference, you have to load the model.

```
# Load the daved model
model = load_model('final_model.h5')
```

Inference

ML Pipeline - Inference



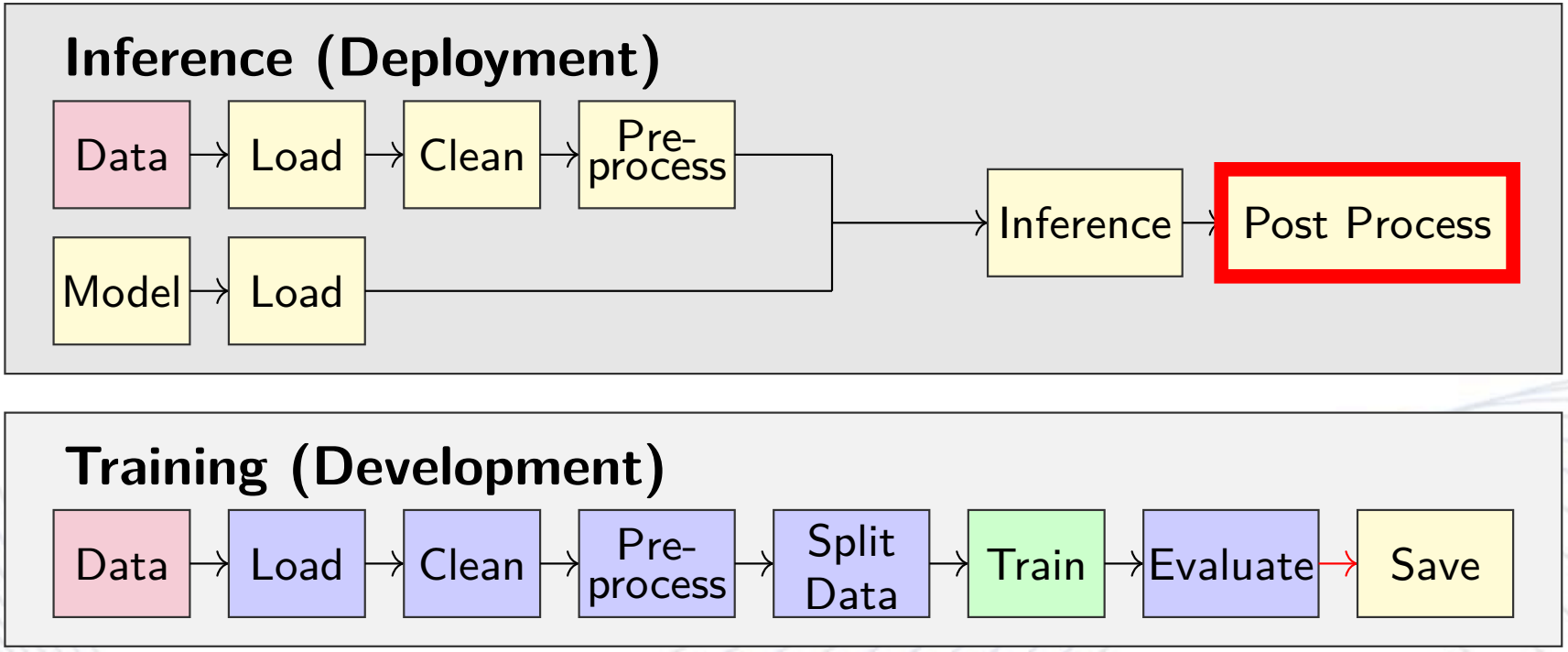
ML Pipeline - Inference

After Loading the model and loading the image, we do the inference.

```
# predict the class
predict_value = model.predict(img)
digit = argmax(predict_value)
```

Post Process

ML Pipeline - Post Process



ML Pipeline - Post Process

In the post process, you can use the result for the next steps.

ML Pipeline - Complete Example - Part I

```
# make a prediction for a new image.
from numpy import argmax
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.models import load_model

# load and prepare the image
def load_image(filename):
    # load the image
    img = load_img(filename, grayscale=True, target_size=(28,
          28))
    # convert to array
    img = img_to_array(img)
    # reshape into a single sample with 1 channel
    img = img.reshape(1, 28, 28, 1)
    # prepare pixel data
    img = img.astype('float32')
    img = img / 255.0
    return img
```

ML Pipeline - Complete Example - Part II

```
# load an image and predict the class
def run_example():
    # load the image
    img = load_image('sample_image.png')
    # load model
    model = load_model('final_model.h5')
    # predict the class
    predict_value = model.predict(img)
    digit = argmax(predict_value)
    print(digit)

# entry point, run the example
run_example()
```

ML Pipeline - Complete Example

Running the example first loads and prepares the image, loads the model, and then correctly predicts that the loaded image represents the digit 7.

Evaluation

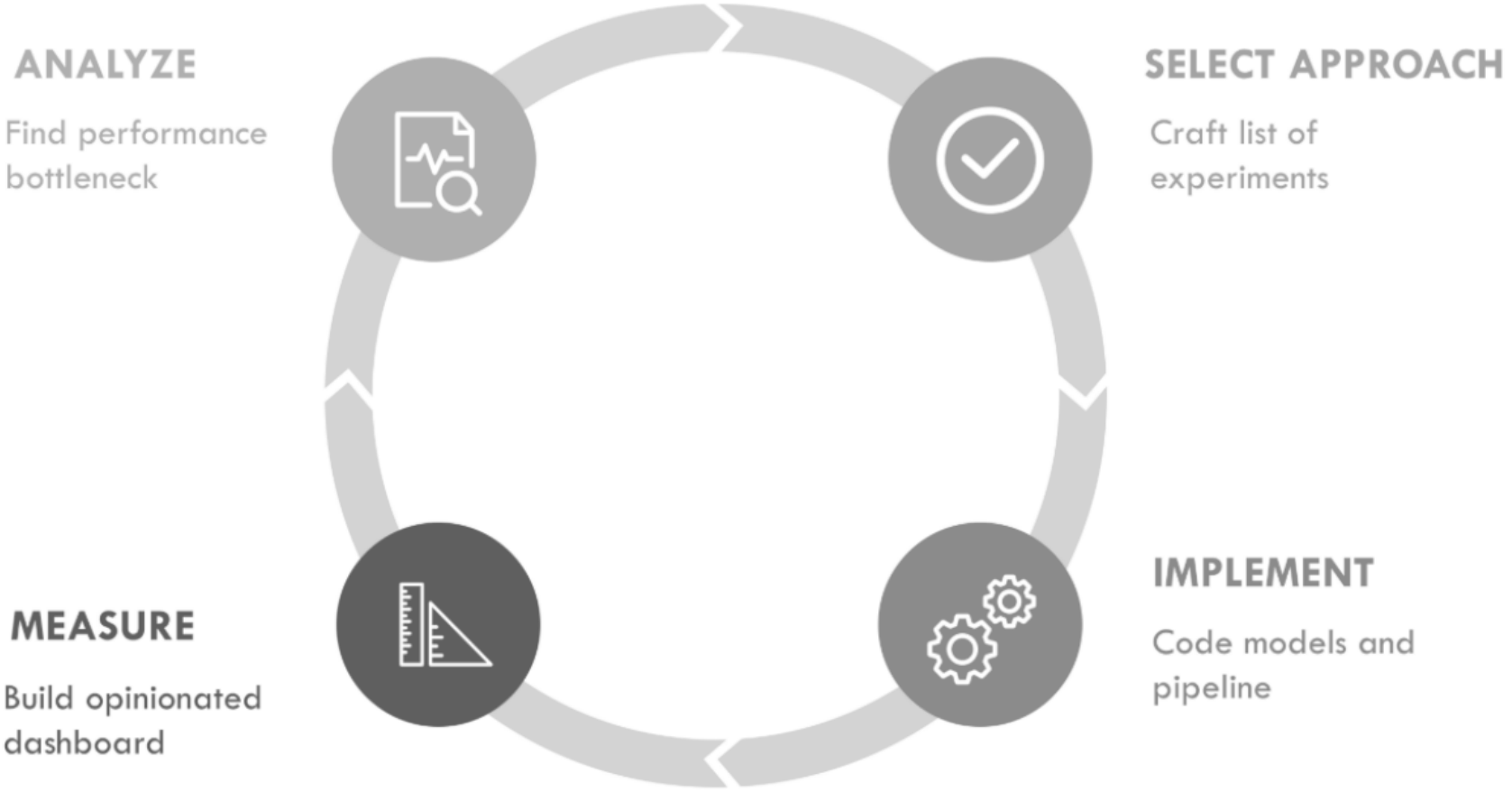
ML Pipeline - Evaluation

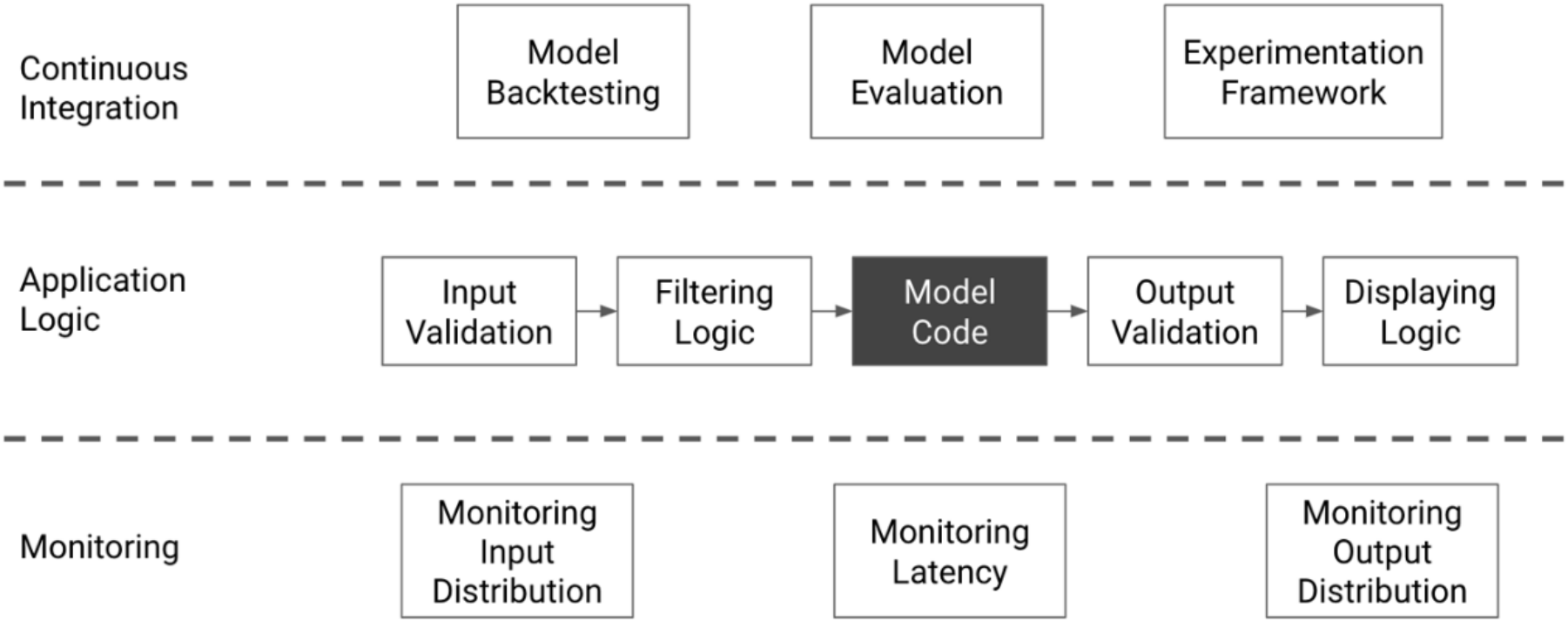
Some ideas for extending the example:

- **Tune Pixel Scaling.** Explore how alternate pixel scaling methods impact model performance as compared to the baseline model, including centering and standardization.
- **Tune the Learning Rate.** Explore how different learning rates impact the model performance as compared to the baseline model, such as 0.001 and 0.0001.
- **Tune Model Depth.** Explore how adding more layers to the model impact the model performance as compared to the baseline model, such as another block of convolutional and pooling layers or another dense layer in the classifier part of the model.

Iterate on Models

Iterate on Models - ML Loop



Deploy and Monitor

Typical Production Modeling Pipeline